

Báo cáo rà soát test code

<!-- framework: JUnit | run_id: run_20260620_184616_522662 -->

Framework: JUnit | Run ID: run_20260620_184616_522662

Tóm tắt kỹ thuật

- Framework: JUnit Jupiter (JUnit 5)
- Kết quả chạy: Tất cả 15 test đều pass, không lỗi biên dịch hay runtime.
- Coverage: 100 % (Jacoco) – đạt ngưỡng chất lượng coverage.
- Vấn đề thực thi: Không có lỗi thực thi, nhưng các yêu cầu về annotation (@TempDir, @ParameterizedTest, @Tag, @ExtendWith, @RegisterExtension) và migrate từ JUnit 4 chưa được thực hiện.
- Trình tự kiểm thử được đánh giá: Tập test BankAccountTest.java so với source BankAccount.java.

Lỗi nghiêm trọng

- Không phát hiện.

Test còn thiếu

- Thiếu test sử dụng @TempDir để kiểm chứng thao tác file tạm (yêu cầu “Kiểm thử thao tác file tạm nếu test cần dùng thư mục tạm”).
- Thiếu test parameterized (@ParameterizedTest + @ValueSource hoặc implicit fallback conversion) cho các getter hoặc các trường hợp khác, như yêu cầu “Kiểm thử implicit fallback argument conversion”.
- Thiếu kiểm thử migrate JUnit 4 → JUnit 5: không có test nào kiểm tra việc thay thế @RunWith, @Rule, @Category bằng các annotation JUnit 5 tương ứng.
- Thiếu test xác thực hành vi checkActive() qua phương thức public deactivate() (theo TC-015 trong test plan). Test hiện tại kiểm tra checkActive() qua deposit(), không đáp ứng yêu cầu “Private checkActive() behavior verified via public method that invokes it (deactivate)”.
- [out of scope]: Không có thông tin về các phương thức nghiệp vụ khác (ví dụ transfer khi target inactive) để kiểm tra implicit conversion hay các trường hợp biên giới khác ngoài những đã có.

Assertion yếu

- Không phát hiện. Các assertion đều kiểm tra giá trị thực tế và thông điệp ngoại lệ một cách cụ thể.

Rủi ro maintainability

- Không có rủi ro đáng kể về flaky test: các test không phụ thuộc vào thời gian, mạng hay hệ thống file ngoại trừ các trường hợp chưa được triển khai (@TempDir).
- Mã test hiện tại không sử dụng mock hoặc extension phức tạp, nên dễ bảo trì.

Gợi ý sửa ngay

1. Thêm test với @TempDir:

@Test

```
void TC_016_writeAndReadTempFile(@TempDir Path tempDir) throws IOException {  
    Path file = tempDir.resolve("data.txt");  
    Files.writeString(file, "a,b,c");  
    String content = Files.readString(file);  
    assertEquals("a,b,c", content);  
}
```

2. Thêm parameterized test cho getter hoặc cho các giá trị biên:

@ParameterizedTest

@ValueSource(strings = {"ACC001", "ACC002", "ACC003"})

```
void TC_017_getAccountNumber_returnsProvidedValue(String acc) {  
    BankAccount accObj = new BankAccount(acc, 0.0);  
    assertEquals(acc, accObj.getAccountNumber());  
}
```

3. Kiểm tra implicit fallback conversion (nếu có factory method/constructor): tạo một class mẫu và viết test tương tự.

4. Thêm test migrate JUnit 4 → JUnit 5: tạo một lớp test mẫu dùng

@RunWith/@Rule/@Category, sau đó viết test xác nhận chúng đã được thay thế bằng @ExtendWith, @RegisterExtension, @Tag.

5. Sửa TC-015 để kiểm tra deactivate() khi tài khoản không active:

@Test

```
void TC_015_deactivateWhenInactiveThrows() {  
    BankAccount account = new BankAccount("ACC999", 0.0);  
    account.deactivate(); // làm tài khoản không active  
    IllegalStateException ex = assertThrows(IllegalStateException.class,  
    account::deactivate, "Phải ném IllegalStateException khi deactivate trên tài khoản không  
    active");  
    assertEquals("Account is inactive", ex.getMessage());  
}
```

(hoặc thêm một phương thức public mới nếu deactivate() không gọi checkActive(); trong trường hợp này cần bổ sung phương thức public để gọi checkActive()).

Thực hiện các cải tiến trên sẽ đáp ứng đầy đủ các yêu cầu trong Requirement JSON và Test Plan, đồng thời nâng cao độ bao phủ và tính duy trì của bộ test.